AGENTIC AI · FINAL PROJECT REPORT

PROJECT MONTAGE

An End-to-End Agentic AI Pipeline for Autonomous Multimedia Production

Submitted By	Maryum Fasih · Areen Zainab
Roll Numbers	22i-0756 · 22i-1115
Presented To	Sir Owais Idrees
Course	Agentic AI
Submission Date	May 2025

1 · ABSTRACT

Generative artificial intelligence has reached a point where individual models can produce compelling text, photorealistic imagery, and natural-sounding speech. What remains elusive is the ability to stitch these capabilities into something cohesive — a unified, temporally consistent multimedia sequence born from a single human instruction. **Project Montage** addresses this gap directly. Built as a multi-phase, graph-based agentic pipeline, the system coordinates specialised LLM and media-generation agents through a centrally managed JSON state manifest that acts as the sole source of truth across all stages of production. The result is a system that can transform a plain-text prompt into a fully synchronised, multi-scene video — while supporting non-destructive, intent-driven editing and deterministic version control at every step.

2 · INTRODUCTION

The promise of generative AI has always been broader than any single modality. Yet, despite the extraordinary advances made in language modelling, image synthesis, and speech generation, the tooling available to practitioners has largely remained siloed. A user who wants to create a short animated sequence from scratch must manually orchestrate a chain of unrelated tools — writing a script in one interface, generating visuals in another, stitching audio in a third — with no shared state connecting them. The output is inevitably inconsistent: characters change appearance between scenes, dialogue is misaligned with on-screen action, and any small revision forces a full restart.

This is not a failure of individual models. It is a failure of architecture. The models themselves are more than capable; what is missing is a coherent orchestration layer that treats multimedia production not as a sequence of isolated API calls, but as a structured, stateful workflow where every decision in Phase 1 implicitly constrains and informs every action in Phase 3.

Project Montage was built to close this gap. Inspired by the way a professional film production is structured — from the writer's room to the studio floor — the system maps each creative function onto a specialised agent, connects them through a shared state manifest, and orchestrates the entire pipeline via a graph-based execution engine. The result is a system where a single sentence of intent becomes a fully produced, synchronised video sequence, with every creative decision traceable, editable, and reversible.

3 · PROBLEM STATEMENT

Three fundamental problems emerge when attempting to automate video production with generative models. Each is technically distinct, yet all three stem from the same root cause: the absence of shared, persistent state across the generation pipeline.

3.1 Identity Drift Across Scenes

Diffusion-based image and video models are stateless by design. Without an explicit mechanism to anchor a character's appearance, the same character rendered in Scene 1 will subtly — and sometimes dramatically — differ from their counterpart in Scene 4. Hair colour, facial structure, and clothing all drift as the stochastic generation process compounds across calls. This makes naive sequential generation unsuitable for narrative content.

3.2 Temporal Desynchronisation

Text-to-speech engines and video generation models operate on entirely different temporal scales. A TTS model produces audio of unpredictable duration; a video model generates clips of a fixed or loosely controlled length. Without a mechanism to measure, communicate, and enforce temporal alignment between these outputs, the resulting content is disjointed — actions complete before dialogue does, or silence stretches where speech should be.

3.3 The Destructive Edit Problem

In the absence of fine-grained state management, altering a single detail — one line of dialogue, one location descriptor — forces a complete regeneration of the entire pipeline. This is prohibitively expensive in both compute and time, and it fundamentally undermines the iterative creative process. A truly usable production tool must support targeted, non-destructive editing at the sub-scene level.

4 · SYSTEM OVERVIEW

At its core, Project Montage is a **state-driven, multi-agent factory**. The system's central innovation is the **Shared State Manifest** — a structured JSON document that every agent in the pipeline reads from and writes back to. Rather than passing raw prompts between models, the system passes this evolving manifest. As the pipeline advances, agents append new metadata — audio durations, image paths, video URIs — ensuring that downstream agents are always implicitly constrained by upstream decisions.

The pipeline is divided into two primary phases — a creative phase and a production phase — separated by a mandatory human approval checkpoint. This separation is intentional: it prevents the system from spending significant compute on media generation before the user has confirmed the narrative and visual direction. A fifth phase handles aesthetic mastering and distribution formatting, elevating the output from a rough assembly of clips to a polished, platform-ready product.

Phase 1	**Writer's Room**	Script · Characters · Reference Portraits
HITL	**User Approval Gate**	Review narrative & visual identity before proceeding
Phase 2	**Studio Floor**	Voice Synthesis · Video Generation · Face-Swap · Lip-Sync
Phase 3–4	**Composition & Assembly**	Audio-video mux · Scene stitching · Timeline sync
Phase 5	**Mastering & Distribution**	Colour grading · Metadata generation · Platform export

Table 1: Pipeline Phase Overview

5 · SYSTEM ARCHITECTURE

5.1 Phase 1 — The Writer's Room

Phase 1 is where intent becomes structure. The **Scriptwriter Agent** receives the user's initial prompt and, constrained by a rigidly defined JSON schema in its system prompt, architects a multi-scene screenplay. The output is not free-form prose — it is a validated data structure containing scene identifiers, location descriptors, character rosters, and dialogue lines each paired with specific visual cues. This forces the model to reason about the narrative spatially and temporally from the outset.

The **Character Designer** then scans this script, isolates named entities, and constructs comprehensive Identity Profiles — detailed descriptors covering physical appearance, clothing, emotional range, and artistic reference style. These profiles are committed to ChromaDB as vector embeddings, making them retrievable across sessions. Finally, the **Image Synthesizer** consumes these profiles to generate high-fidelity reference portraits — the visual anchors that all downstream video generation is constrained by.

5.2 Phase 2 — The Studio Floor

Phase 2 operates entirely from the finalized Phase 1 manifest and is designed for maximum parallelism. LangGraph's branching capabilities allow every scene to be treated as an independent sub-graph, with audio and video generation running concurrently rather than sequentially.

The **Voice Synthesis Agent** processes each dialogue line using Edge-TTS, generating WAV files and — critically — recording the precise millisecond duration of each audio clip back into the manifest. The **Video Generation Agent** reads this duration data alongside the visual cues and character portraits, either querying an AI video model or retrieving stock footage via the Pexels API, then uses FFmpeg to hard-trim the visual output to match the audio length exactly. **Face-Swap and Lip-Sync Agents** complete the loop, mapping the Phase 1 reference portraits onto video subjects and synchronising lip movements to the synthesised phoneme stream.

5.3 Phase 3–4 — Composition & Assembly

The orchestrator handles deterministic muxing of the generated assets — overlaying audio on the trimmed video clips, stitching individual scenes into a coherent sequence, and producing per-scene outputs that feed into the mastering phase. Because the manifest contains precise timestamps for every asset, this assembly step is fully deterministic: the same manifest always produces the same composite.

5.4 Phase 5 — Aesthetic Mastering & Distribution

Phase 5 serves as the system's mastering suite — the post-production layer that elevates a technically correct assembly of clips into a visually unified, platform-ready product. It was designed specifically to address the visual 'seam problem' that appears when a pipeline draws from multiple generation backends simultaneously.

The **Aesthetic Mastering Agent** analyses the luminosity, contrast, and colour temperature of every scene in the compiled sequence. When a project mixes diffusion-generated video with stock footage — the common case in rapid prototyping mode — the visual registers of those clips can clash noticeably. The agent applies programmatic colour-grading LUTs (Look-Up Tables) to normalise each shot, ensuring that a darkly lit, synthetically generated apartment scene sits coherently next to a stock exterior without the jarring tonal shift that would otherwise signal a production shortcut.

Distribution is handled with equal automation. The **Metadata Generation** sub-agent reads the narrative payload of the scene manifest and synthesises SEO-optimised titles, descriptions, and tag clouds formatted for platforms such as YouTube and Vimeo. **Format Adaptation** uses FFmpeg to re-render the final sequence into multiple aspect ratios — 9:16 for TikTok and Reels, 16:9 for traditional displays — without requiring a manual re-edit. Finally, **Project Archiving** commits the complete asset set and full version history to a permanent project vault in the vector store, enabling future retrieval, remixing, and continuation of any project across sessions.

6 · METHODOLOGY

The methodological foundation of Project Montage rests on four principles: **semantic translation**, **human-in-the-loop validation**, **tool-agnostic invocation**, and **localised state projection**. Together, they ensure the pipeline is both robust and adaptable.

Semantic Translation

The pipeline begins by translating abstract human intent — a sentence, a mood, an idea — into a machine-actionable data structure. The Scriptwriter Agent is heavily constrained by its system prompt to output only validated JSON matching a predefined schema. This is not a stylistic choice; it is a correctness requirement. Downstream agents cannot operate on ambiguous prose. By forcing the model to commit to concrete scene identifiers, character names, and temporal boundaries at this stage, the system eliminates the possibility of hallucinated or malformed outputs propagating through the pipeline.

Human-in-the-Loop Checkpoint

A mandatory approval gate sits between Phase 1 and Phase 2. The frontend dashboard surfaces the generated script, character profiles, and reference portraits for user review before any computationally expensive media generation begins. This is a deliberate architectural choice: it is far cheaper to revise a JSON manifest than to regenerate video. The checkpoint also establishes a baseline of user consent — the system will not proceed without explicit approval.

Tool Discovery via MCP

Rather than hard-coding API integrations, agents in the Montage ecosystem use the **Model Context Protocol (MCP)** to discover and invoke tools at runtime. When the Video Generation Agent needs a clip, it queries the MCP server for available tools, discovers the relevant schema, and invokes it. This decoupling is architecturally significant: adding a new video backend — say, a different diffusion model — requires only registering a new tool with the MCP server. The agent logic itself requires zero modification.

State Projection

Passing the full manifest — with all character profiles, scene data, edit history, and audio metadata — to every agent would quickly exhaust the token budget of any reasonably-sized LLM. Instead, the orchestration graph implements **State Projection**: each agent receives only the heavily pruned, scene-scoped slice of the manifest that is directly relevant to its task. The Scene 2 Voice Synthesizer sees only Scene 2's dialogue and emotional cues. This not only preserves token budget but measurably improves reasoning quality by eliminating irrelevant context.

7 · KEY FEATURES

7.1 Graph-Based Agentic Execution

The entire pipeline is modelled as a **LangGraph StateGraph** — a directed graph where nodes represent agents and edges represent state transitions. Unlike linear procedural code, a graph-based model supports conditional branching, cyclical revision loops, and parallel sub-graphs natively. Scene generation in Phase 2 exploits this directly: every scene is an independent branch, executing its audio and video generation concurrently, reducing total pipeline latency proportionally to the scene count.

7.2 Modular, Provider-Agnostic Architecture

No single model or API is hard-wired into the pipeline. The cognitive core switches seamlessly between Groq (Llama-3) and Google Gemini based on environment configuration, ensuring availability and cost management. The video generation layer toggles between stock footage retrieval and AI-generated video via a single environment flag. This modularity means the system can adapt to new models as they emerge without structural refactoring.

7.3 Persistent Vector Memory

ChromaDB is not used purely as a logging mechanism. Character Identity Profiles and state manifests are committed as vector embeddings, giving the system the ability to recall stylistic decisions across execution sessions. This addresses a common failure mode in agentic systems: context loss between runs. A character designed in one session remains consistently described in subsequent sessions without requiring the user to re-specify anything.

7.4 Real-Time Frontend Orchestration

The React/Vite frontend is not a passive display layer. It serves as the primary HITL interface, surfacing the evolving manifest in a readable format, accepting edit instructions via natural language input, and communicating with the FastAPI backend asynchronously. Pipeline events — agent transitions, asset generation completions, error states — are streamed to the UI in real time, giving the user genuine visibility into the system's execution.

8 · EDIT AGENT & VERSION CONTROL

The Edit Agent is arguably the most consequential component in the system from a user-experience perspective. It reframes the role of AI in the production workflow: not a one-shot generator to be re-run from scratch on every change, but a **collaborative creative partner** that accepts ongoing instruction and applies changes surgically.

8.1 Intent-Driven State Mutation

When a user submits a revision — *"Make Alex sound more panicked in Scene 2"* or *"Change the location of Scene 1 to a neon-lit alleyway"* — the Edit Agent parses the natural language intent, locates the precise JSON node in the active manifest that corresponds to the requested change, and mutates only that value. The rest of the manifest — and the already-generated assets for unaffected scenes — remains entirely untouched. This is **Delta Execution**: the orchestration graph evaluates the state difference between the previous and current manifest, identifies which downstream assets are invalidated, and queues only those for re-generation.

8.2 Checkpointing and Undo

Every mutation is preceded by a state snapshot. Before the Edit Agent writes any change to the active manifest, the current JSON state is serialised and committed to ChromaDB with a sequential version tag. This creates a complete, traversable edit history for the project.

If a change yields undesirable results, the user issues an "undo" command. The system retrieves the most recent committed snapshot from ChromaDB, re-establishes it as the active manifest, and instantly restores the project to its prior configuration. Because previously approved media assets are cached and referenced by path in the manifest — not re-generated — the restoration is immediate. There is no recompute cost to reversing a change. The undo stack is not shallow: the system supports infinite traversal of the version history, allowing a user to step back through multiple iterations non-destructively.

9 · TECHNOLOGY STACK

Architectural Layer	Core Technologies	Purpose
State Orchestration	LangGraph, Python 3.11+	Manages the StateGraph, phase transitions, and parallel branching

Architectural Layer	Core Technologies	Purpose
API & Backend	FastAPI, Uvicorn	High-performance async REST interface bridging UI to the graph
Tooling Protocol	Model Context Protocol (MCP)	Standardises tool discovery and invocation for autonomous agents
Cognitive Core	Groq API (Llama-3), Google Gemini	Drives narrative reasoning, structuring, and intent parsing
Vector Memory	ChromaDB	State persistence, versioning checkpoints, and identity profiles
Media Processing	FFmpeg, OpenCV, Edge-TTS	Audio synthesis, video trimming, lip-sync, and asset composition
Client Interface	React, Vite, Tailwind CSS	Real-time interactive dashboard for pipeline monitoring and HITL

Table 2: Core Technology Stack by Architectural Layer

10 · PHASE 5: AESTHETIC MASTERING & DISTRIBUTION

Phase 5 was the final architectural addition to Project Montage and arguably the most consequential from an output-quality standpoint. Its premise is simple: a pipeline that produces technically synchronised but visually incoherent media is not production-ready. Phase 5 closes that gap.

10.1 The Aesthetic Mastering Agent

When a project draws from both AI-generated video and stock footage — which is the default in rapid-prototyping mode — the footage originates from fundamentally different rendering pipelines with different colour science, gamma curves, and exposure characteristics. Left unaddressed, the resulting sequence reads as a collage rather than a coherent production.

The Mastering Agent resolves this by performing a programmatic colour pass over the entire compiled sequence. It analyses the luminosity histogram, average colour temperature, and contrast ratio of each scene. It then computes and applies per-scene colour-grading **LUTs (Look-Up Tables)** to normalise the visual register across all clips. A scene generated in a synthetically rendered dark interior will, after mastering, sit tonally consistent with a bright outdoor stock clip — the viewer experiences a continuous film, not a patchwork of mismatched renders.

10.2 Automated Metadata Generation

Traditional post-production requires a separate manual effort to produce the promotional and indexing assets that accompany a video release. Phase 5 automates this entirely. The Metadata Generation sub-agent reads the finalised scene manifest — which by this point contains the complete narrative arc, character names, locations, and emotional beats — and synthesises SEO-optimised titles, long-form descriptions, and tag clouds tailored to the target platform. A project destined for YouTube receives a different metadata profile than one packaged for Vimeo, with the differences in tone and keyword density handled automatically.

10.3 Format Adaptation & Project Archiving

A single composition rarely suits every distribution context. The Format Adaptation step uses FFmpeg to transcode and re-crop the final sequence into multiple aspect ratios in a single pass: **9:16** for TikTok and Instagram Reels, **16:9** for YouTube and traditional broadcast, and **1:1** for feed-native placements. The crop is not a naive centre-cut; it is informed by the visual cue data in the manifest, which identifies the primary subject of each frame.

Finally, Project Archiving commits the complete, finalised asset bundle — video files, audio tracks, character profiles, the full version history of the state manifest, and all generated metadata — to a permanent project vault in ChromaDB. This enables a use case that is otherwise impossible in generative pipelines: returning to a completed project weeks later, making a targeted edit via the Edit Agent, and re-exporting only the affected scenes without losing any previously approved work.

Sub-Component	Input	Output
Aesthetic Mastering Agent	Compiled scene sequence (mixed sources)	Colour-graded, visually unified video sequence
Metadata Generation	Finalised scene manifest (narrative payload)	SEO-optimised titles, descriptions, tag clouds per platform
Format Adaptation	Master video file + visual cue data	Multi-aspect-ratio exports (9:16, 16:9, 1:1)
Project Archiving	All assets + complete version history	Permanent project vault entry in ChromaDB

Table 3: Phase 5 Sub-Component Summary

11 · CHALLENGES & SOLUTIONS

Building a coherent multi-agent pipeline surfaced a set of challenges that are distinct from those encountered in single-model applications. The following table documents the most significant

problems encountered and the architectural decisions made to resolve them.

Challenge	Root Cause	Solution Implemented
State Amnesia Across Scenes	Diffusion models have no inherent memory between inference calls	Centralised JSON manifest locks character descriptors as immutable anchors for every downstream generation call
Temporal Audio-Video Drift	TTS and video generation operate on independent temporal scales	Voice Synthesis agent returns precise millisecond durations; Video agent uses these values to hard-trim clips before muxing
Context Window Exhaustion	Full manifest passed to every agent quickly saturates token budgets	State Projection — agents receive only a pruned, scene-scoped slice of the manifest, shielding them from irrelevant global context
Latency from Diffusion Video Models	Cloud-based video models introduce multi-minute latency and rate limits	Hybrid mode: Pexels API for rapid prototyping; diffusion model enabled only after structure is approved via HITL checkpoint
Destructive Editing Cycles	Re-generating full output for a single change wastes compute and time	Delta Execution — Edit Agent mutates only the relevant JSON node; only the affected downstream assets are re-rendered

Table 4: Key Engineering Challenges and Implemented Solutions

12 · RESULTS & OUTPUT

The system produces multi-scene video sequences — complete with synchronised dialogue, consistent character appearance, and coherent visual pacing — from a single input prompt in a matter of minutes. In stock footage mode, a three-scene sequence with three speaking characters can be fully produced in under five minutes; in AI video generation mode, the same sequence takes longer due to cloud model latency, but the structural pipeline remains identical.

Three output qualities stand out as particularly significant. First, **predictable correctness**: by forcing the creative process through validated JSON schemas at Phase 1, the system eliminates the class of errors caused by malformed or hallucinated LLM outputs breaking downstream rendering tools. The pipeline either produces a valid output or fails with a structured, inspectable error — never a corrupt asset.

Second, **genuine editability**: the version-controlled Edit Agent transforms the production workflow from a series of one-shot generations into an iterative creative process. Users can refine tone, adjust pacing, change locations, and revise dialogue without ever restarting from scratch. Third,

production velocity: tasks that would require hours of manual effort across multiple professional tools — scriptwriting, casting, storyboarding, voiceover, video sourcing, and assembly — are compressed into a single, guided session lasting minutes.

13 · CONCLUSION

Project Montage makes a clear architectural argument: the path to reliable automated media production runs through structure, not raw model capability. The models available today — for text, image, audio, and video — are individually impressive. What they lack is the scaffolding needed to make their outputs interoperable, consistent, and correctable. By providing that scaffolding in the form of a shared state manifest, a graph-based orchestration engine, and a version-controlled edit layer, this system demonstrates that coherent multi-modal production is achievable right now, with existing models.

Beyond the technical outcome, the project exposed a broader insight about agentic system design: the quality of a multi-agent pipeline is determined less by the capability of individual agents and more by the quality of the contracts between them. The JSON schema enforced at Phase 1 is not an implementation detail — it is the backbone of the entire system. Investing in that contract design is what makes everything downstream reliable.

The system stands as a proof-of-concept with genuine production utility, and its modular architecture positions it well for the expansions outlined in the following section.

14 · FUTURE WORK

The current implementation handles linear narrative construction efficiently, but several directions present clear opportunities for meaningful expansion:

Real-Time Composition Engines

Replacing the static FFmpeg composition step with a real-time rendering engine — Unreal Engine's MetaHuman pipeline, or a web-native WebGL canvas — would allow instantaneous playback of edits without rendering delays. This would bring the Edit Agent workflow closer to the real-time feedback loop of a traditional non-linear editor.

Spatial Audio Reasoning

The Voice Synthesis and Mastering agents currently treat audio as flat stereo output. Extending them to understand spatial context — dynamically applying reverb for locations tagged INT. CAVE, or attenuation for characters at distance — would significantly improve the cinematic quality of the

audio mix.

Branching Narrative Support

The state manifest's tree structure could be extended to support non-linear, branching storylines. Rather than a single sequence of scenes, the manifest would encode conditional branches, enabling the automated generation of interactive media, choose-your-own-adventure content, and dynamic video game cutscenes — all from the same orchestration architecture.

Finer-Grained Character Consistency

While the Identity Profile system significantly reduces drift, integrating LoRA fine-tuning or IP-Adapter conditioning on the reference portraits at generation time would bring character consistency to a much higher standard, approaching the level required for professional animated content.

Project Montage · Agentic AI · FAST-NUCES Islamabad · 2025